

10. Perceptron multicamadas

No século XVII, Leibniz criou o primeiro computador mecânico do mundo. Desde então, criar máquinas capazes de realizar cálculos aritméticos com agilidade foi um fetiche para muitos matemáticos. Mais de dois séculos depois, diversos esforços culminariam na criação de um modelo formal para computação e de computadores digitais.

A ambição de criar máquinas inteligentes levou diversos pesquisadores a procurar inspiração na biologia para criar *redes neurais artificiais*. Nesse capítulo, faremos um curto passeio pela literatura e discutiremos redes neurais multicamadas, popularmente conhecidas como perceptron multicamadas (*multilayer perceptron*, MLP).

10.1 Neurônios artificiais e o perceptron de Rosenblatt

Neurônios possuem terminações nervosas (i.e. dendritos) que capturam estímulos elétricos. Por sua vez, dado estímulo suficiente, um neurônio pode propagar um sinal elétrico a partir de seus axônios. Há mais de 75 anos, McCulloch e Pitts (1943) se inspiraram nessa estrutura para propor um modelo simples de *neurônio artificial*, que ainda hoje é o elemento basilar de diversas redes neurais artificiais modernas.

Esse neurônio artificial recebe estímulos como um vetor $\mathbf{x} \in \mathbb{R}^d$ — intuitivamente, contando com uma dimensão por terminação nervosa. Então, o neurônio combina os estímulos $\mathbf{x}$ baseado na importância de cada dendrito como $\mathbf{x} \cdot \mathbf{w}$. Finalmente, o neurônio soma um estímulo constante b e aplica uma função de ativação $h : \mathbb{R} \rightarrow \{-1, 1\}$, que decide que sinal o neurônio deve emitir. Nesse contexto, b pode ser interpretado como um nível de energia intrínseco ao neurônio. Quanto maior b , menor a quantidade de estímulo $\mathbf{x} \cdot \mathbf{w}$ necessária para que h dispare 1. Seguindo a convenção dos capítulos anteriores, nossa discussão abstrairá a presença de b , já que sempre podemos incluir uma componente constante ao vetor de entrada $\mathbf{x}$ sem perder generalidade.

Uma década após McCulloch e Pitts, Rosenblatt (1957) mostrou como esse modelo de neurônio poderia ser implementado em um computador para aproximar funções binárias $f : \mathbb{R}^d \rightarrow \{-1, 1\}$. O método de Rosenblatt, amplamente conhecido como **Perceptron**, utiliza a função sinal como ativação e possui um processo específico para o aprendizado de $\mathbf{w}$. Mais concretamente, a saída $h(\mathbf{x})$ de um perceptron é dada por:

$$h(\mathbf{x}) = \text{sign}(\mathbf{x}_n^\top \mathbf{w}) = \begin{cases} +1 & \text{se } \mathbf{x} \cdot \mathbf{w} > 0 \\ -1 & \text{caso contrário} \end{cases}. \quad (10.1)$$

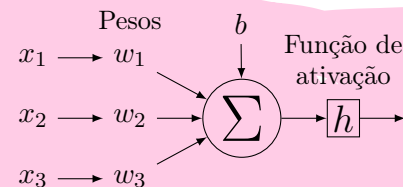


Figura 10.1: Neurônio artificial.

Seguindo a equação acima, $h(\mathbf{x})$ é 1 se e somente se o ângulo entre $\mathbf{x}$ e $\mathbf{w}$ é menor que 90° . Caso um exemplo $(\mathbf{x}, \mathbf{y})$ tenha sido incorretamente classificado com $h(\mathbf{x}) = 1$, podemos deslocar $\mathbf{w}$ na direção oposta a $\mathbf{x}$ (i.e., $\mathbf{y}\mathbf{x}$) para diminuir o valor de $\mathbf{x} \cdot \mathbf{w}$. O raciocínio complementar se aplica quando o exemplo é incorretamente classificado com $h(\mathbf{x}) = -1$. Baseado nessa intuição, Rosenblatt propôs um algoritmo heurístico de aprendizado que percorre várias vezes o conjunto de treino, identificando cada exemplo incorreto $(\mathbf{x}, \mathbf{y})$ e incrementando $\mathbf{w}$ com $\mathbf{y}\mathbf{x}$. Alguns anos depois, Novikoff (1962) provou que o algoritmo de Rosenblatt converge no caso linearmente separável — desde que algumas condições de regularidade sejam satisfeitas.

Apesar de simples, o perceptron foi motivo de excitação na comunidade de IA, sendo alvo de vários trabalhos e base para outros métodos. Uma variação notável é o *adaline* (Widrow & Lehr, 1990) que minimiza diretamente o resíduo quadrático entre as pré-ativações e os rótulos de treinos usando gradiente descendente — de maneira muito similar à que costumamos treinar modelos de regressão atualmente. Apesar do momento gerado pelo perceptron, a pesquisa *mainstream* em IA passou paulatinamente a se focar em métodos baseados em lógica matemática durante os anos 60. Nesse contexto, Minsky e Papert (1969) mostraram que redes como o perceptron e adaline não conseguem aprender algumas funções muito simples. O exemplo clássico é o operador lógico XOR (ou exclusivo). Essas limitações não são surpreendentes dada a natureza linear dessas redes e, mesmo assim, já se sabia que era possível tornar redes mais expressivas combinando neurônios de maneira composicional. De todo modo, Minsky e Papert (1969) martelou os pregos do caixão, jogando a pesquisa em redes neurais no ostracismo por 10+ anos.

Algoritmo 2 Treino do Perceptron

Recebe: $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$
 $\mathbf{w} \leftarrow \mathbf{0}$
for $t = 1, 2, \dots$ **do**
 for $n = 1, 2, \dots, N$ **do**
 $\hat{y}_n \leftarrow \text{sign}(\mathbf{w}^\top \mathbf{x}_n)$
 if $y_n \neq \hat{y}_n$ **then**
 $\mathbf{w} \leftarrow \mathbf{w} + y_n \mathbf{x}_n$
 end if
end for
end for

Figura 10.2: Algoritmo de aprendizado para o perceptron de Rosenblatt.

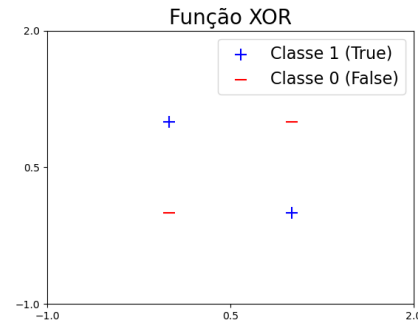


Figura 10.3: Função XOR. Não existe uma função linear capaz de separar os exemplos positivos dos negativos.

10.2 Perceptron multicamadas

Depois de ≈ 15 de inverno, Rumelhart et al. (1986) mostrou que era possível aprender os pesos de uma rede neural com mais que uma camada usando gradiente descendente. A arquitetura proposta por Hinton é baseada em camadas. Cada camada l contém p_l perceptrons paralelos equipados com a função de ativação sigmoide, que conhecemos bem por sua aplicação em regressão logística. Por sua vez, cada perceptron na camada $l + 1$ recebe as saídas dos perceptrons da camada anterior l . A Figura 10.4 ilustra essa arquitetura multicamadas, que ficou amplamente conhecida como perceptron multicamadas (*multilayer perceptron*, MLP). De maneira sucinta, podemos descrever recursivamente a saída de um MLP $\Psi^{(L)}$ com $L > 0$ camadas como:

$$\Psi^{(l)}(\mathbf{x}) = \begin{cases} \phi_l(\Psi^{(l-1)}(\mathbf{x})\mathbf{W}_l) & \text{if } l > 0 \\ \mathbf{x} & \text{otherwise} \end{cases}$$

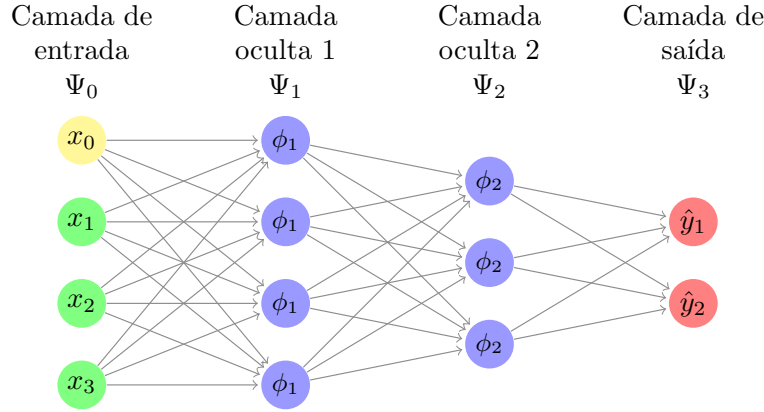


Figura 10.4: Representação gráfica de um MLP para regressão. As arestas denotam pesos. Na última camada, cada saída $\hat{y}_i$ é o resultado de uma combinação linear das saídas da camada oculta. Nesse caso, pode-se dizer que a função de ativação da última camada é a identidade. Em casos de classificação binária, poderíamos ter uma saída apenas e aplicar a função sigmoide. Outra opção é manter ambas as saídas e usar uma softmax para fazer as probabilidades de ambas as classes somarem 1. A última opção também se aplica no caso de saídas categóricas. Note que a ilustração omite os bias (intercept) das camadas ocultas.

onde ϕ_l e $\mathbf{W}_l$ denotam a função de ativação e a matriz de pesos da camada l . Nesse contexto, abusamos de notação usando ϕ_l como uma função que age em cada componente de seu argumento de entrada.

Para aprender os pesos $\mathbf{W}_1, \dots, \mathbf{W}_L$ Hinton propôs usar um método que ficou conhecido como *backpropagation*— que é usado até hoje. Já nessa época era comum usar a cross-entropia binária/categórica como loss para problemas de classificação binária/múltipla, o MSE para problemas de regressão. Em poucas palavras, o backpropagation é uma implementação eficiente da regra da cadeia (para derivadas) para calcular os gradientes da loss com respeito aos pesos do MLP. Para abordar o backpropagation, deixe-nos primeiro simplificar notação definindo $\mathbf{z}^{(l)} = \Psi^{(l)}(\mathbf{x})$. Note então que a saída do nosso MLP para um vetor de entrada $\mathbf{x}$ pode ser descrita como:

$$\mathbf{z}^{(l)} = \phi_l(\dots \phi_2(\phi_1(\mathbf{x}\mathbf{W}_1)\mathbf{W}_2)\dots \mathbf{W}_l). \quad (10.2)$$

Tratando cada $\mathbf{W}_\ell$ como um vetor coluna (que pode ser obtido 'desenrolando' a matriz linha-a-linha), note que o gradiente da nossa loss ℓ de escolha com relação à matriz de pesos $\mathbf{W}_j$ é igual à Jacobiana transposta de ℓ , que pode ser descrita como:

$$J_{\mathbf{W}_j}^{\ell(\mathbf{z}^{(L)}, y)} = J_{\mathbf{z}^{(L)}}^{\ell(\mathbf{z}^{(L)}, y)} J_{\mathbf{z}^{(L-1)}}^{\mathbf{z}^{(L)}} J_{\mathbf{z}^{(L-2)}}^{\mathbf{z}^{(L-1)}} \dots J_{\mathbf{z}^{(j)}}^{\mathbf{z}^{(j+1)}} J_{\mathbf{W}_j}^{\mathbf{z}^{(j)}}, \quad (10.3)$$

para qualquer camada $j > 0$. O importante aqui é nota que a Equação 10.3 nos revela um fato computacionalmente útil: a Jacobiana para os pesos de uma camada j depende diretamente das Jacobianas das camadas subsequentes.

Baseado nessa observação sobre a regra da cadeia, backpropagation computa gradientes/Jacobianas em dois passos: o *forward* e o *backward*. No forward, propagamos cada vetor de input $\mathbf{x}$ iterativamente através das camadas, guardando as matrizes Jacobianas $J_{\mathbf{z}^{(l-1)}}^{\mathbf{z}^{(l)}}$ e $J_{\mathbf{W}^{(l)}}^{\mathbf{z}^{(l)}}$.

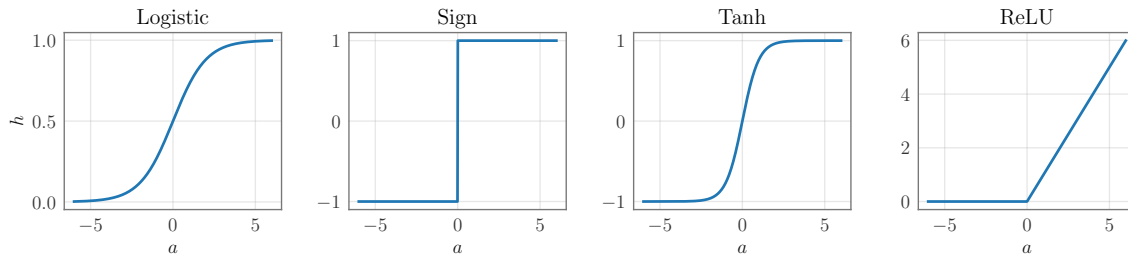


Figura 10.5: Funções de ativação não-lineares. Embora produzam valores em intervalos diferentes, as funções logística e tangente hiperbólica são diferenciáveis. A função sinal fornece saídas iguais a -1 ou 1, enquanto a função ReLU é não-limitada e produz valores em $[0, \infty]$.

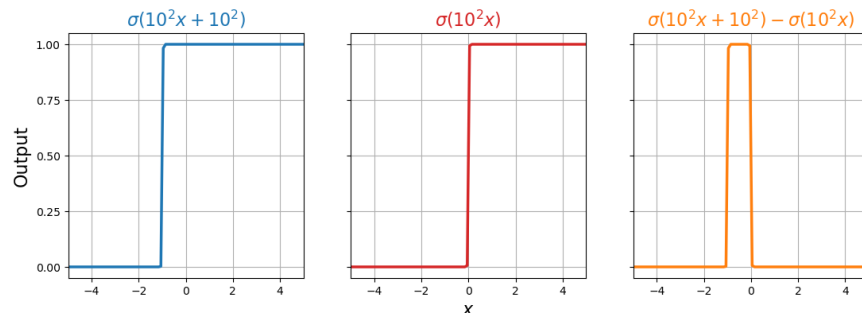
Na última camada, computamos também $J_{\mathbf{z}^{(L)}}^\ell$. O passo backward para um único exemplo de treinamento $\mathbf{x}$ pode ser descrito da seguinte maneira:

1. $\zeta \leftarrow J_{\mathbf{z}^{(L)}}^\ell(\mathbf{z}^{(L)}, y)$
2. Para $\tilde{l} = L, \dots, 1$ em ordem decrescente:
 - (a) $\mathbf{W}_l^{\text{new}} \leftarrow \mathbf{W}_l - \alpha \left(\zeta J_{\mathbf{W}_l}^{\mathbf{z}^{(l)}} \right)^\top$
 - (b) Se $\tilde{l} > 1$, $\zeta \leftarrow \zeta J_{\mathbf{z}^{(l-1)}}^{\mathbf{z}^{(l)}}$

onde $\mathbf{W}_l^{\text{new}}$ denota o valor atualizado da matriz de peso $\mathbf{W}_l$ e α é a taxa de aprendizado do gradiente descendente. Note que esse algoritmo pode ser facilmente adaptado para comportar vários exemplos. Para tal, basta somar-se gradientes para cada exemplo de treinamento. Além disso, o algoritmo é facilmente paralelizável. Além disso, a mesma ideia pode ser utilizada com outros métodos de otimização por gradiente, como SGD ou Adam.

Apesar de Rumelhart et al. (1986) utilizarem ativações sigmoide nas camadas $1, \dots, L-1$, existem patologias por trás dessa escolha. Mais especificamente as partes chapadas (quase constantes) da função sigmoide possuem gradiente próximo de zero, o que pode fazer alguns pesos não serem atualizados durante o treino. O mesmo vale para a tangente hiperbólica, que é fortemente ligada à sigmoide — $\sigma(t) = 1/2 + 1/2 \tanh(t/2)$. Por muito tempo, essa patologia impôs um desafio para treinar MLPs profundas com $L > 3$ camadas. A solução viria a ser utilizar uma ativação chamada *rectified linear unit* (ReLU), de formato $\phi(t) = \max(0, t)$, que fora proposta por (Fukushima, 1975) — mas apenas popularizadas décadas depois por Nair e Hinton (2010). A Figura 10.5 ilustra as funções de ativações que vimos até aqui.

Mais que três camadas? É bem conhecido que redes neurais com apenas uma camada oculta já são aproximadores universais (Cybenko, 1989; Hornik et al., 1989). A intuição por trás desse tipo de resultado (para o caso escalar) é que podemos construir funções com valores não-zero em partes arbitrárias do domínio somando a saída de neurônios. Por sua vez, podemos combinar essas funções simples linearmente para aproximar (até algum grau de precisão) funções arbitrariamente intrincadas. Podemos, por exemplo, combinar linearmente deus neurônios sigmoide para obter uma função aproximadamente constante por partes, como ilustrado na figura abaixo.



Resta então a pergunta: para que mais camadas? Na prática, aproximar funções arbitrárias com uma única camada oculta pode implicar usar uma quantidade proibitiva de neurônios. Por exemplo, Telgarsky (2016) prova que existem MLPs camadas que não podem ser bem aproximadas por redes de uma cada, a não ser que as últimas possuam uma quantidade exponencial de neurônios. Intuitivamente, quando empilhamos camadas, as funções que combinamos linearmente antes de cada ativação podem se tornar progressivamente mais intrincadas. Não surpreendentemente, treinar arquiteturas cada vez mais profundas é um fetiche bem enraizado na comunidade de redes neurais.

Bibliografia

- Ahn, K., Yun, C., & Sra, S. (2020). SGD with shuffling: optimal rates without component convexity and large epoch requirements. *Advances in Neural Information Processing Systems (NeurIPS)* (ver página 11).
- Atkeson, C. G., Moore, A. W., & Schaal, S. (1997). Locally Weighted Learning. *Artificial Intelligence Review*, 11(1–5) (ver página 20).
- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press. (Ver página 22).
- Broomhead, D., & Lowe, D. (1988). Multivariable Functional Interpolation and Adaptive Networks. *Complex Systems*, 2, 321–355 (ver página 30).
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27 (ver página 19).
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4), 303–314 (ver página 59).
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14, 179–211 (ver página 64).
- Fukushima, K. (1975). Cognitron: A Self-Organizing Multilayer Neural Network. *Biological Cybernetics*, 20, 121–136 (ver página 59).
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366 (ver página 59).
- Lloyd, S. P. (1982). Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28, 129–137 (ver página 30).
- Manning, C. D., & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press. (Ver página 22).
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4), 115–133 (ver páginas 5, 56).
- Micchelli, C. A. (1986). Interpolation of scattered data: Distance matrices and conditionally positive definite functions. *Constr. Approx.*, 2, 11–22 (ver página 30).
- Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press. (Ver página 57).
- Mohamed, S., Rosca, M., Figurnov, M., & Mnih, A. (2020). Monte Carlo Gradient Estimation in Machine Learning. *Journal of Machine Learning Research (JMLR)* (ver página 42).
- Nair, V., & Hinton, G. (2010). Rectified linear units improve restricted boltzmann machines. *International conference on machine learning (ICML)* (ver página 59).
- Nguyen, L., Tran-Dinh, Q., Phan, D., Nguyen, P., & van Djik, M. (2021). A Unified Convergence Analysis for Shuffling-Type Gradient Methods. *Journal of machine learning research (JMLR)* (ver página 11).
- Novikoff, A. B. (1962). On Convergence Proofs on Perceptrons. *Proceedings of the Symposium on the Mathematical Theory of Automata*, 12, 615–622 (ver página 57).

- Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian processes for machine learning*. MIT Press. (Ver página 52).
- Rosenblatt, F. (1957). *The perceptron: A perceiving and recognizing automaton (Project PARA)* (relatório técnico). Cornell University (Technical Report 85-460-1). (Ver página 56).
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *nature*, 323(6088), 533–536 (ver páginas 57, 59).
- Safran, I., & Shamir, O. (2021). Random Shuffling Beats SGD Only After Many Epochs on Ill-Conditioned Problems. *Advances in Neural Information Processing Systems (NeurIPS)* (ver página 11).
- Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning - From Theory to Algorithms*. Cambridge University Press. (Ver página 44).
- Telgarsky, M. (2016). benefits of depth in neural networks. Em V. Feldman, A. Rakhlin & O. Shamir (Editores), *COLT*. (Ver página 60).
- Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, 49, 433–460 (ver página 5).
- Weinberger, K. Q., Blitzer, J., & Lawrence, S. K. (2006). Distance Metric Learning for Large Margin Nearest Neighbor Classification. Em Y. Weiss, B. Schölkopf & J. C. Platt (Editores), *Advances in Neural Information Processing Systems 18* (páginas 1473–1480). MIT Press. (Ver página 22).
- Widrow, B., & Lehr, M. (1990). 30 years of adaptive neural networks: perceptron, Madaline, and backpropagation. *Proceedings of the IEEE*, 78(9), 1415–1442. <https://doi.org/10.1109/5.58323> (ver página 57).
- Xu, M., Quiroz, M., Kohn, R., & Sisson, S. A. (2019). Variance reduction properties of the reparameterization trick. *Artificial Intelligence and Statistics (AISTATS)* (ver página 42).